WIPRO CAPSTONE PROJECT REPORT

Goibibo Train Booking Automation

Selenium Pytest POM and Behave BDD Automation Documentation

[PRIYANSHU PRAKASH]

SuperSet ID:- [4467004] • [Selenium with Pytest Batch B2] •

Report Item	Details
Application Under Test	Goibibo Trains
Application URL	https://www.goibibo.com/trains/
Frameworks Covered	Selenium Python Pytest POM and Behave BDD with Gherkin
Reporting Tool	Allure Reports with screenshots and logs
Prepared For	Capstone Automation Project Evaluation
Report Date	24 May 2026

1. Common Project Overview

This capstone project automates the Goibibo train booking workflow using Selenium WebDriver with Python. The same business domain is implemented in two frameworks: Selenium Pytest Page Object Model and Behave BDD with Gherkin. Both validate train search, filters, sorting, negative validations, passenger details, and safe payment form reachability.

The project intentionally stops at test payment form validation and does not complete a real transaction. Allure reports, screenshots, and logs are used as execution evidence.

2. Common Business Scope

- Open Goibibo trains module.
- Select source and destination station codes.
- Select journey date from calendar.
- Search trains and verify results page.
- Apply coach class, departure time, and station filters.
- Validate departure and arrival sorting.
- Validate booking-window and no-direct-train negative cases.
- Proceed through train selection, IRCTC ID, passenger details, contact details, and payment form entry.

3. Common Tools and Technologies

Tool / Concept	Usage in Project
Python	Programming language used for both automation frameworks.
Selenium WebDriver	Automates browser interaction with Goibibo trains.
Page Object Model	Separates locators and web actions from test logic.
Allure Reports	Generates detailed execution reports with steps, screenshots, and logs.
Logging	Maintains execution history in automation.log.
Screenshots	Captures important page states and failure evidence.
Explicit Waits	Synchronizes browser actions without hard-coded time.sleep.

4. Common High-Level Architecture

Layer	Purpose
Test / Feature Layer	Contains Pytest test cases or BDD Gherkin scenarios.
Step / Test Logic Layer	Calls page object methods and performs assertions.
Page Object Layer	Contains reusable page actions for homepage, results page, details page, and payment page.
Utility Layer	Handles config reading, screenshots, logging, CSV/Excel reading where applicable.
Report Layer	Stores Allure results, screenshots, logs, and generated reports.

PART 1: SELENIUM PYTEST POM FRAMEWORK

5. Selenium Framework Description

The Selenium Pytest project is the code-driven automation implementation. It uses Pytest test functions, fixtures, parametrization, CSV data, Excel data, and Allure Pytest annotations. This framework is useful for technical regression execution because each validation is written as a Python test and can be run independently or as a full suite.

5.1 Selenium Project Structure

```
Capstone_Goibibo_Trains_Selenium
├── config
│   └── config.properties
├── data
│   ├── booking_data.xlsx
│   ├── filter_class.csv
│   ├── filter_time.csv
│   ├── filter_station.csv
│   ├── invalid_date.csv
│   ├── no_trains.csv
│   ├── sort_arrival.csv
│   └── sort_departure.csv
├── pages
│   ├── homepage.py
│   ├── results_page.py
│   ├── details_page.py
│   ├── payment_page.py
│   └── basepage.py
├── tests
│   ├── test_cases.py
│   └── test_e2e.py
├── utils
├── conftest.py
├── pytest.ini
└── requirements.txt
```

5.2 Selenium Framework Components

Component	Purpose
conftest.py	Creates WebDriver fixture, opens base URL, applies browser options, and captures failure screenshots.
test_cases.py	Contains positive filter/sort tests and negative validation tests using CSV data.
test_e2e.py	Contains full booking flow using Excel workbook data.
pages/homepage.py	Handles source, destination, date, search, and popup actions.
pages/results_page.py	Handles result validation, class/time/station filters, sorting, and train selection.
pages/details_page.py	Handles IRCTC ID, passenger details, contact details, and proceed-to-payment.
pages/payment_page.py	Handles card tab and test payment form entry.
utils/	Handles config, logging, screenshots, CSV, and Excel reading.

5.3 Selenium Test Coverage

Test Case	Type	Validation
test_coach_class_filter	Positive	Applies class filter and validates matching coach class code.
test_departure_time_filter	Positive	Validates departure times inside selected time window.
test_departure_station_filter	Positive	Validates train departure station code.
test_sort_by_departure_time	Positive	Validates ascending departure time sorting.
test_sort_by_arrival_time	Positive	Validates ascending arrival time sorting.

test_booking_window_limit	Negative	Validates booking-not-open message for invalid future date.
test_absolute_empty_search_route	Negative	Validates no direct trains message.
test_complete_train_booking_flow	End-to-End	Validates route search, train selection, passenger details, and payment form.

5.4 Selenium Sample Code Snippets

Sample Pytest fixture from conftest.py:

```
@pytest.fixture(scope="function")
def driver():
    browser = ConfigReader.get("browser").strip().lower()
    base_url = ConfigReader.get("base_url").strip()
    edge_options = EdgeOptions()
    edge_options.add_argument("--start-maximized")
    edge_options.add_argument("--disable-notifications")
    driver = webdriver.Edge(options=edge_options)
    driver.implicitly_wait(5)
    driver.get(base_url)
    yield driver
    driver.quit()
```

Sample Selenium Pytest validation:

```
@pytest.mark.parametrize("data", class_data)
@allure.feature("Positive Testing")
def test_coach_class_filter(driver, data):
    search_for_trains(driver)
    results_page = ResultsPage(driver)
    assert results_page.verify_results_loaded()
    results_page.apply_dynamic_class_filter(data['ClassFilterLabel'])
    assert
results_page.verify_class_present_in_results(data['ClassExpectedCode'])
```

5.5 Selenium Execution Commands

```
cd Capstone_Selenium_with_Pytest\Capstone_Goibibo_Trains_Selenium
pytest
pytest tests/test_cases.py
pytest tests/test_e2e.py
allure serve allure-results
```

5.6 Selenium Screenshots and Evidence

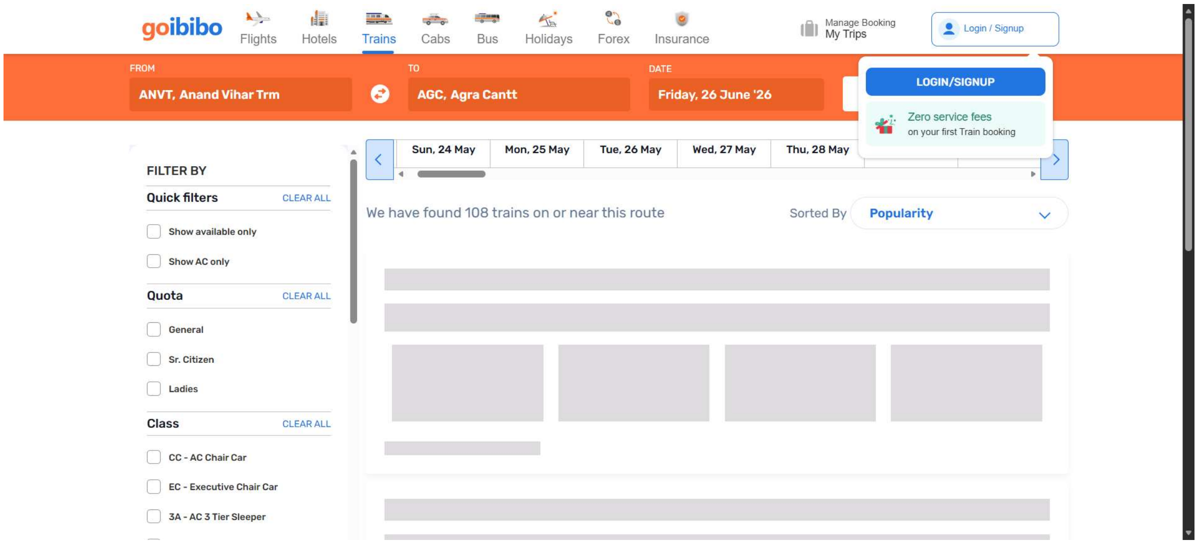

Figure S1: Selenium Pytest evidence - train results page loaded.

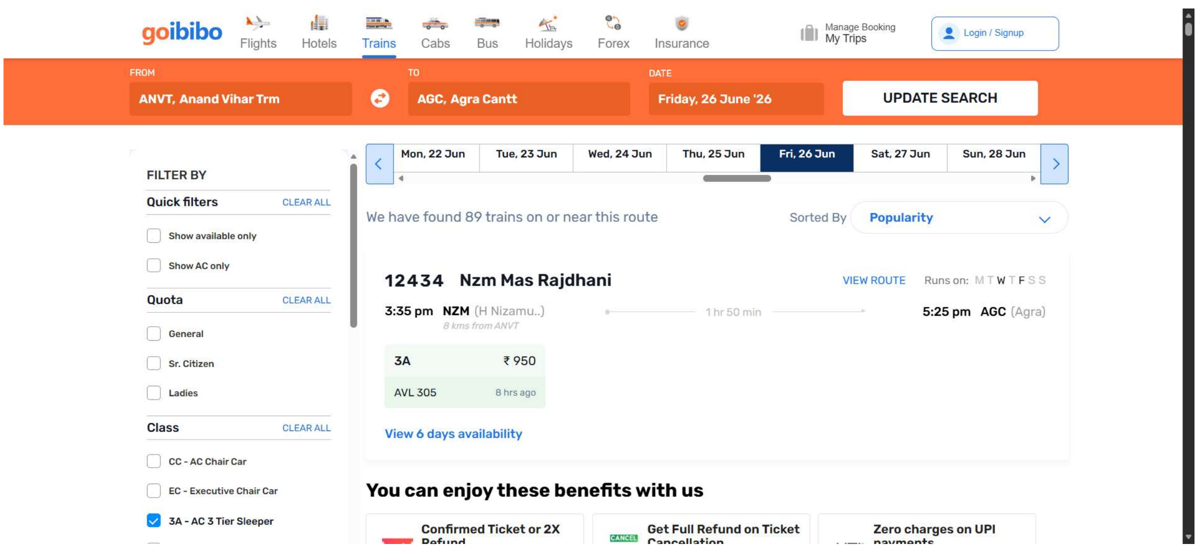

Figure S2: Selenium Pytest evidence - coach class filter applied.

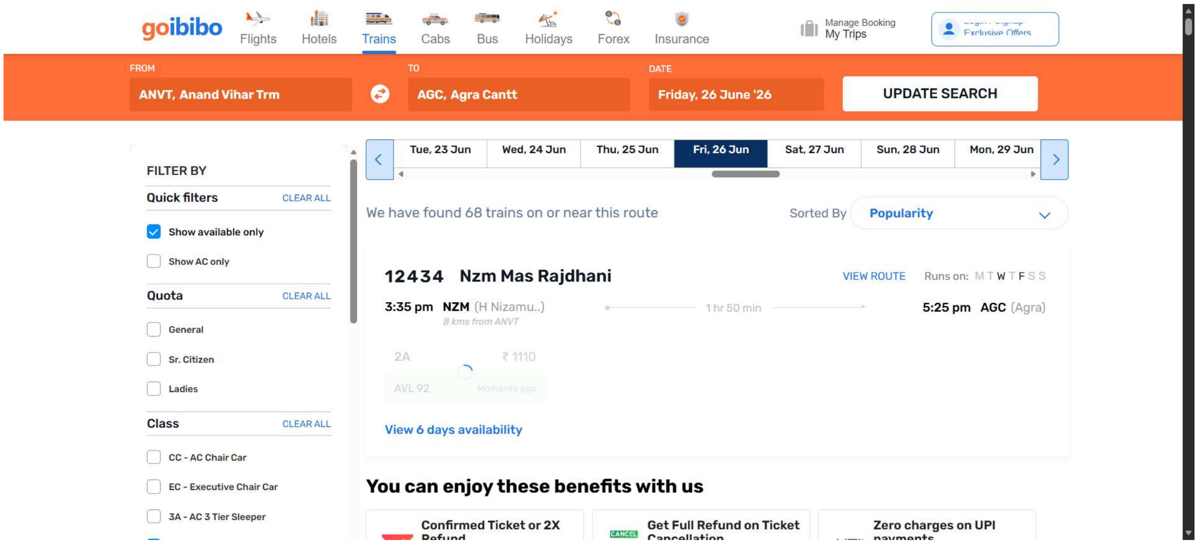

Figure S3: Selenium Pytest evidence - valid 2A train selected.

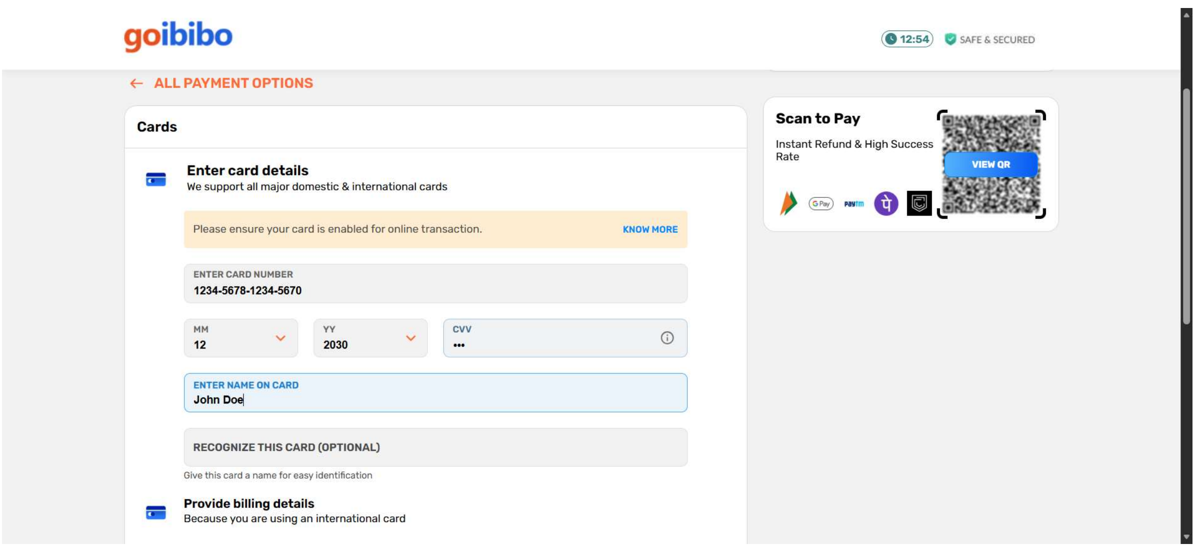

Figure S4: Selenium Pytest evidence - payment form populated.

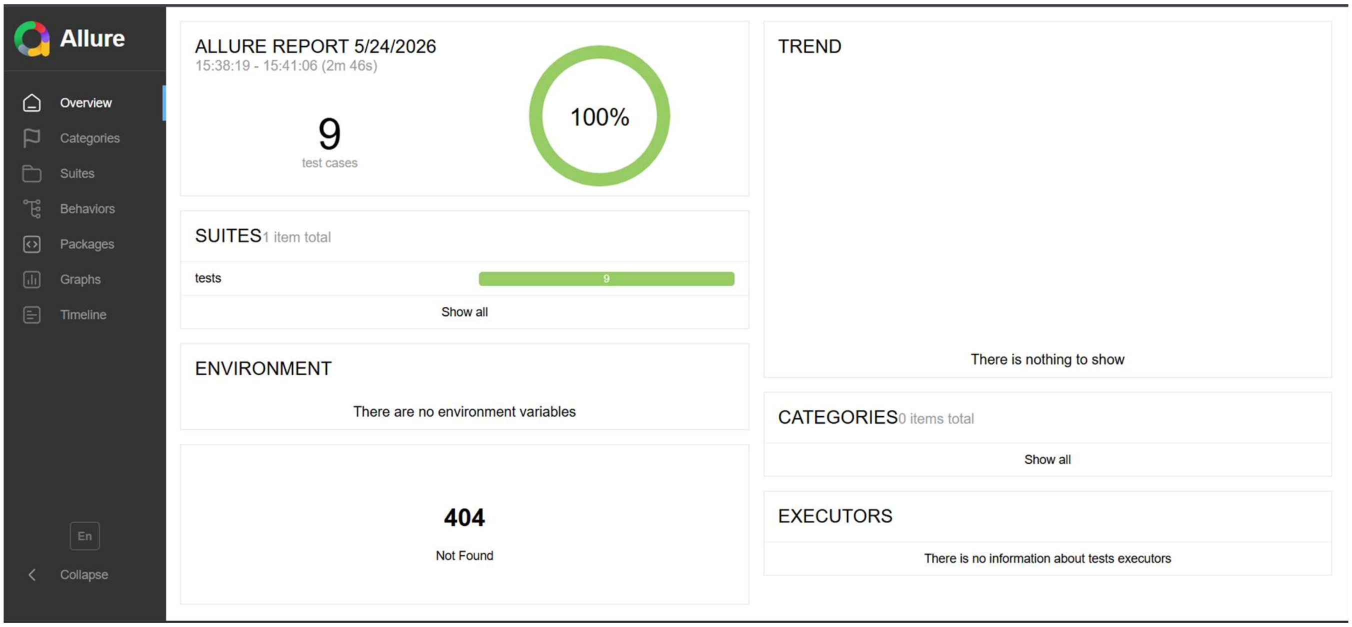

Figure S5: Selenium Pytest Allure evidence – Allure Dashboard Overview.

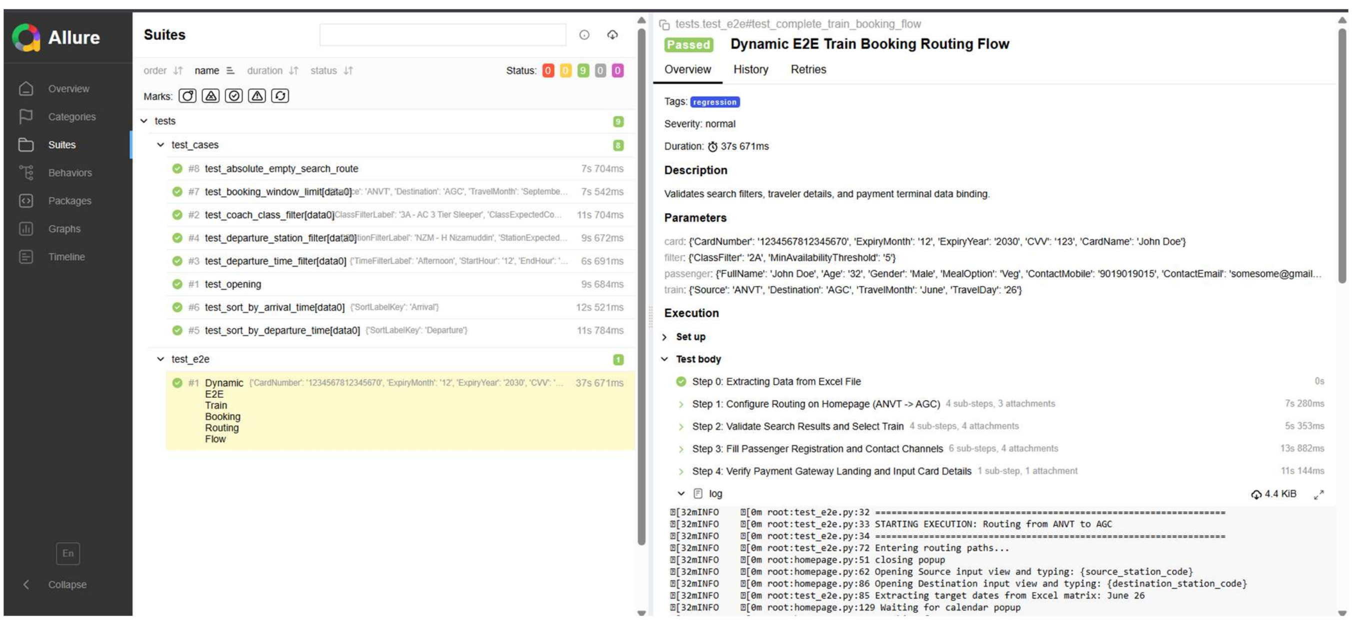

Figure S6: Selenium Pytest Allure evidence – Test Suits with Steps and Screenshots.

5.7 Selenium Execution Summary

Metric	Result
Stored Allure passed test results	9
Stored Allure failed test results	0
Stored Allure broken test results	0
Overall Selenium status	Passed based on available Allure result files

PART 2: BEHAVE BDD FRAMEWORK

6. BDD Framework Description

The Behave BDD project converts the same train automation into business-readable Gherkin scenarios. The important improvement in the BDD project is that test data is written directly inside Scenario Outline example tables instead of using CSV or Excel files. This makes the test intent and test data visible to both technical and non-technical reviewers.

6.1 BDD Project Structure

```
Capstone_Goibibo_Trains
├── config
│   └── config.properties
├── features
│   ├── trains_test_cases.feature
│   ├── environment.py
│   └── steps
│       └── trains_test_cases.py
├── pages
├── utils
├── reports
│   ├── allure-results
│   ├── allure-report
│   └── screenshots
├── behave.ini
├── requirements.txt
└── runtests.py
```

6.2 BDD Framework Components

Component	Purpose
features/trains_test_cases.feature	Contains Gherkin scenarios and Scenario Outline example data.
features/steps/trains_test_cases.py	Maps Given-When-Then steps to Selenium page object methods.
features/environment.py	Starts browser before scenario, captures screenshots/logs, and closes browser after scenario.

runtests.py	Runs Behave with Allure formatter, cleans old results, supports tags, and can serve the Allure report.
pages/	Reuses Page Object Model actions from the Selenium implementation.
reports/screenshots	Stores screenshots from scenario execution.

6.3 BDD Scenario Coverage

Scenario	Tags	Validation
Coach class filter shows matching train classes	@positive @filter @class	Visible train cards show expected class.
Departure time filter keeps trains inside selected window	@positive @filter @time	Departure times remain within selected hour range.
Departure station filter shows trains from selected station	@positive @filter @station	Visible trains depart from expected station code.
Train results can be sorted by departure time	@positive @sort @departure	Departure times are sorted ascending.
Train results can be sorted by arrival time	@positive @sort @arrival	Arrival times are sorted ascending.
Booking outside allowed window shows validation message	@negative @booking-window	Booking-not-open message is displayed.
Routes with no direct trains show no trains message	@negative @no-trains	No direct trains message is displayed.
Complete train booking flow reaches payment form	@regression @e2e	Booking flow reaches payment form and fills card fields.

6.4 BDD Sample Gherkin and Step Code

<u>Sample Gherkin Scenario Outline:</u>

```
@positive @filter @class
Scenario Outline: Coach class filter shows matching train classes
  When I search trains from "<Source>" to "<Destination>" on "<TravelMonth>"
"<TravelDay>"
  Then train results should be displayed
```

```
  When I apply coach class filter "<ClassFilterLabel>"
  Then visible train cards should show class "<ClassExpectedCode>"

Examples:
  | Source | Destination | TravelMonth | TravelDay | ClassFilterLabel
| ClassExpectedCode |
  | ANVT   | AGC         | June        | 26        | 3A - AC 3 Tier Sleeper
| 3A                |
```

<u>Sample step definition:</u>

```
@when('I apply coach class filter "{class_filter}"')
def step_apply_class_filter(context, class_filter):
    with allure.step(f"Apply coach class filter: {class_filter}"):
        ResultsPage(context.driver).apply_dynamic_class_filter(class_filter)

@then('visible train cards should show class "{expected_code}"')
def step_verify_class_filter(context, expected_code):
    assert
ResultsPage(context.driver).verify_class_present_in_results(expected_code)
```

6.5 BDD Execution Commands

```
cd Capstone_BDD_with_Behave\Capstone_Goibibo_Trains
python runtests.py
python runtests.py --tags=@class
python runtests.py --tags=@positive
python runtests.py --tags=@negative
python runtests.py --tags=@e2e
python runtests.py --serve
allure serve reports/allure-results
```

6.6 BDD Screenshots and Evidence

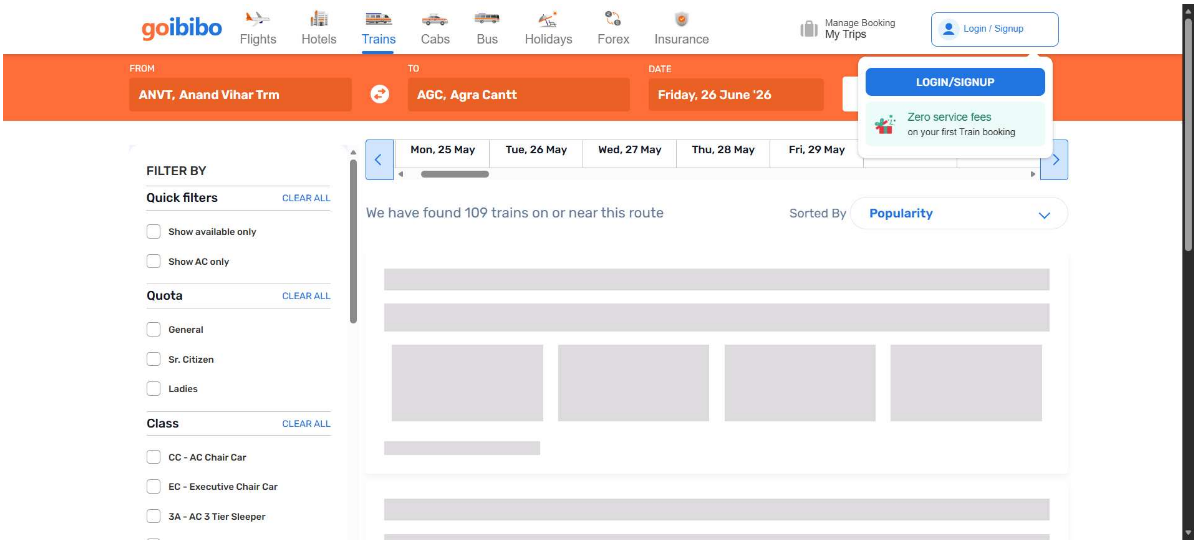

Figure B1: BDD evidence - train results page loaded.

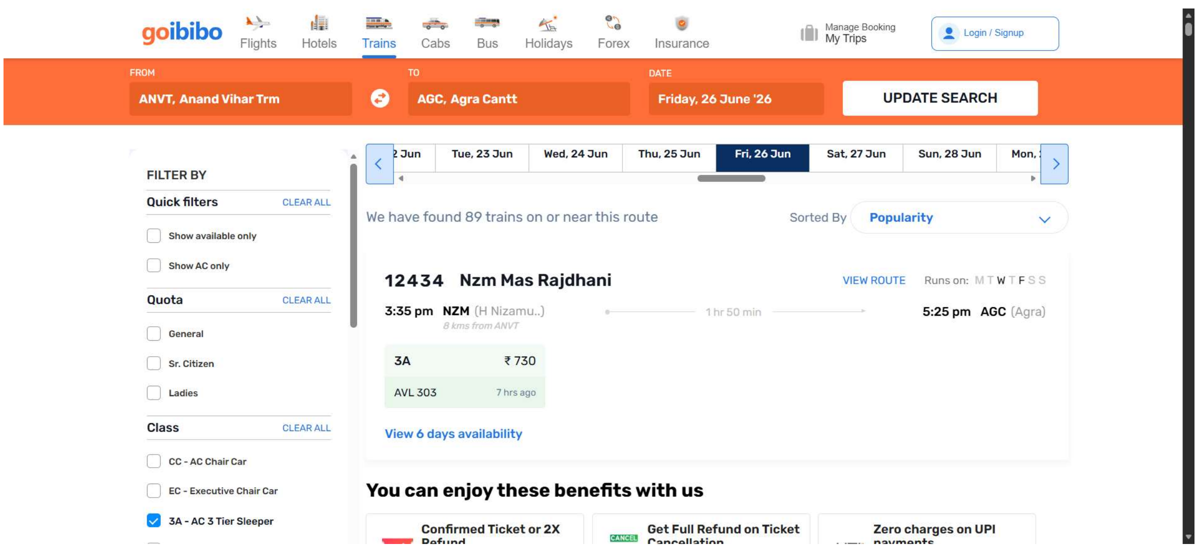

Figure B2: BDD evidence - coach class filter applied.

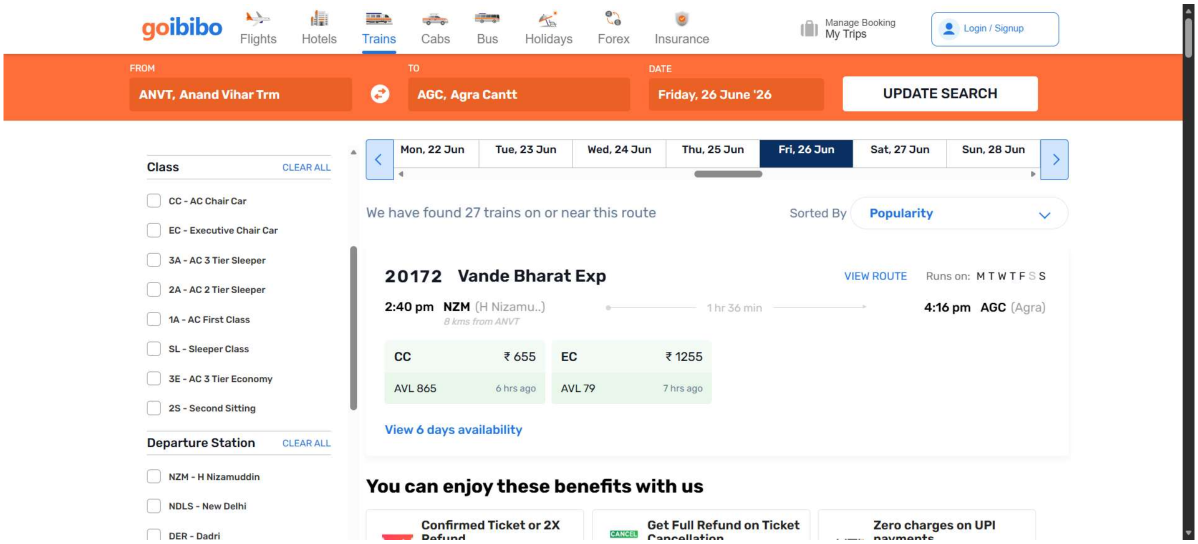

Figure B3: BDD evidence - departure time filter applied.

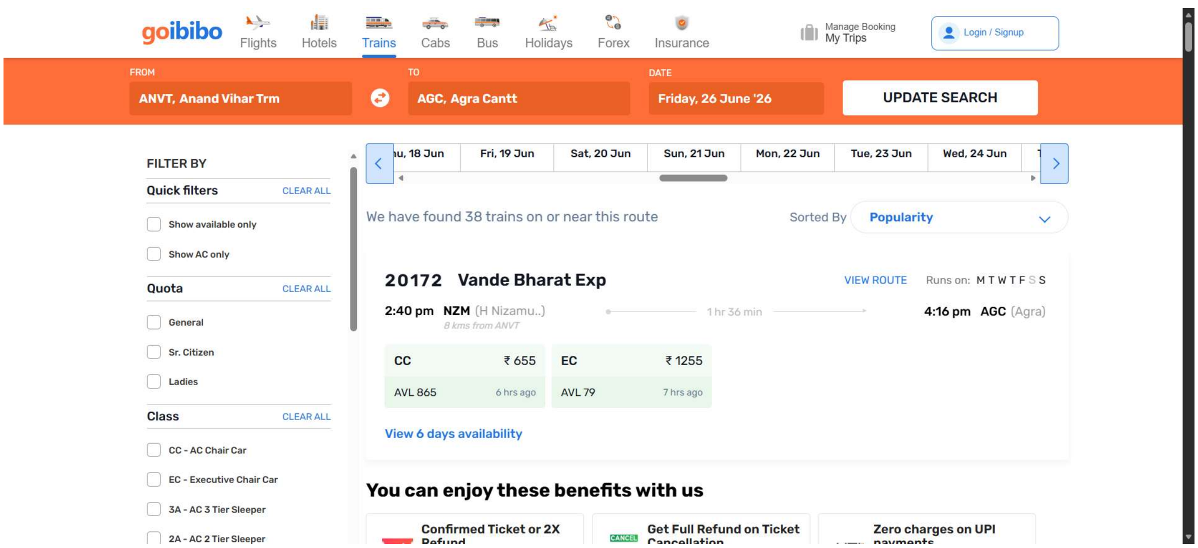

Figure B4: BDD evidence - station filter applied.

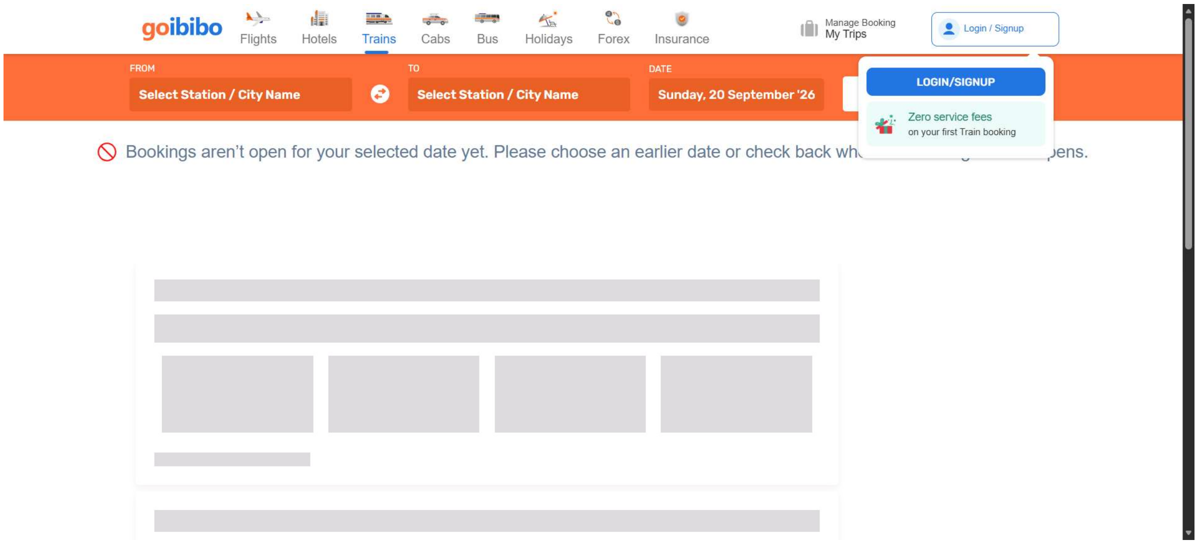

Figure B5: BDD evidence - booking window negative validation.

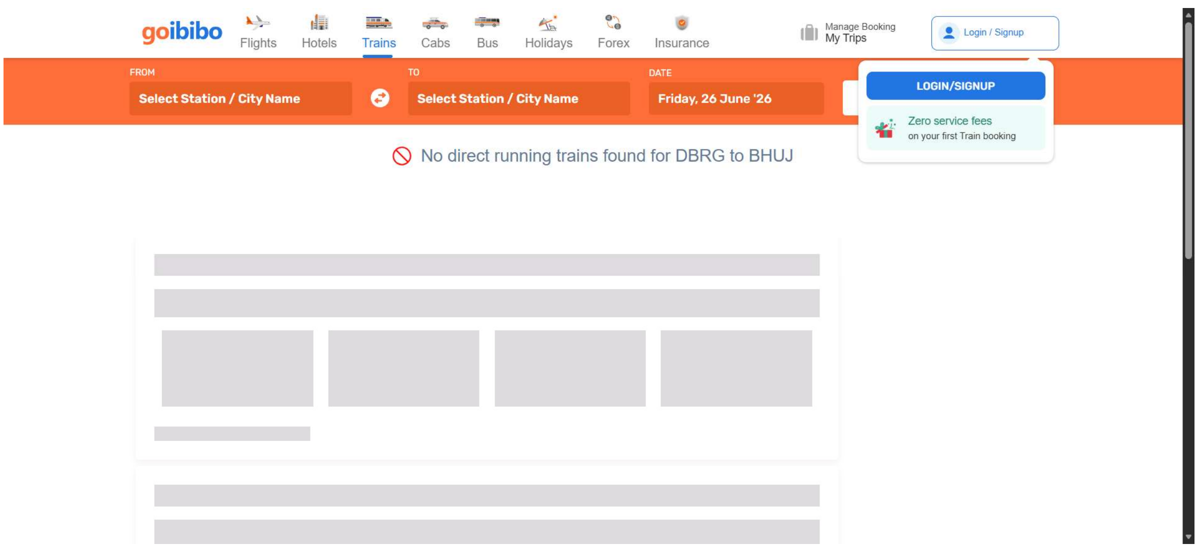

Figure B6: BDD evidence - no direct trains negative validation.

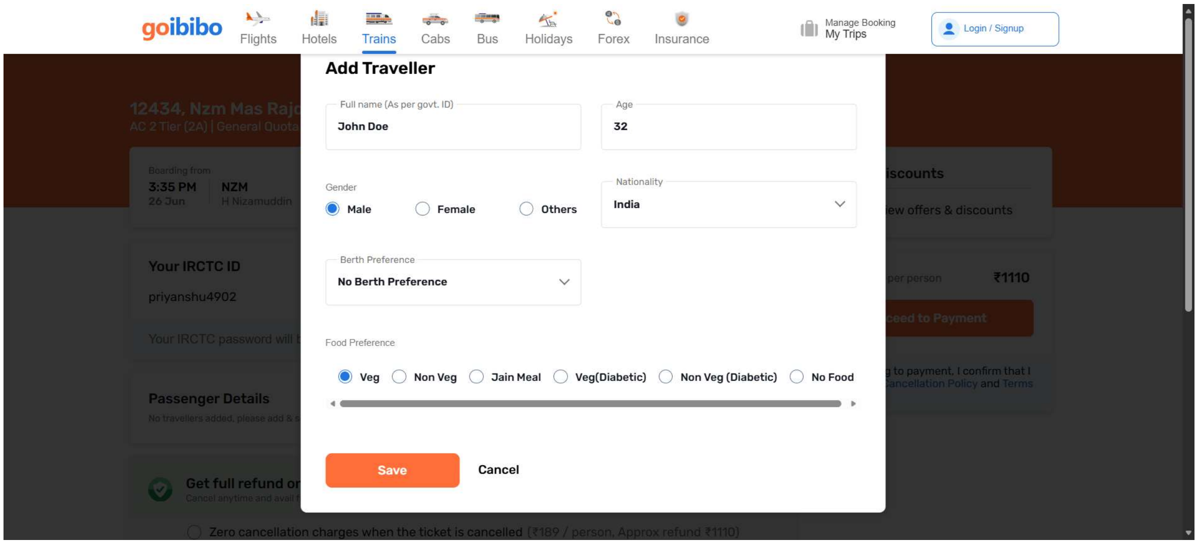

Figure B7: BDD evidence - passenger details populated.

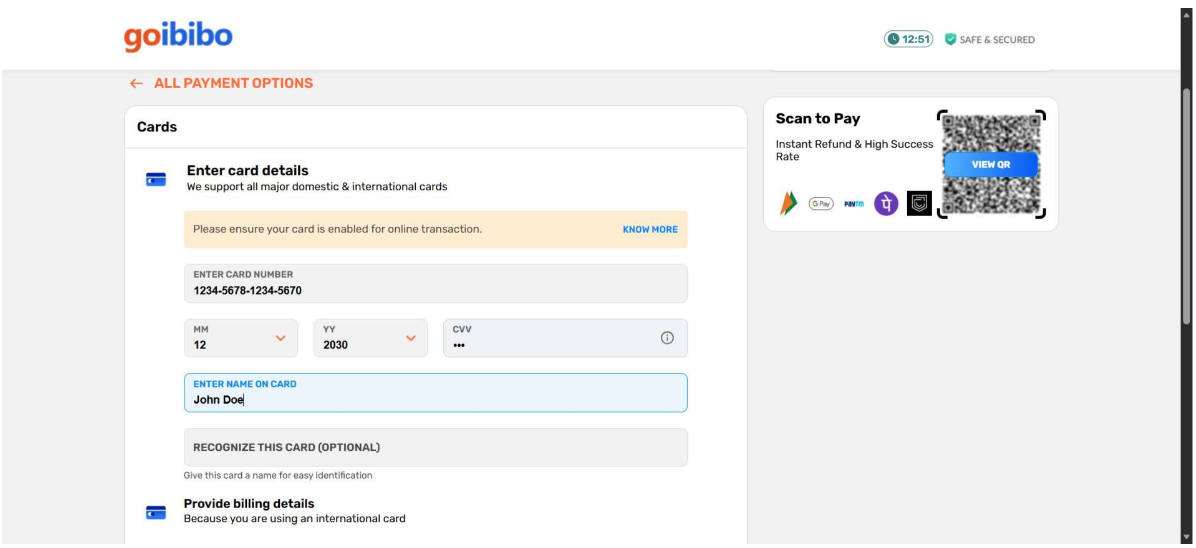

Figure B8: BDD evidence - payment form populated.

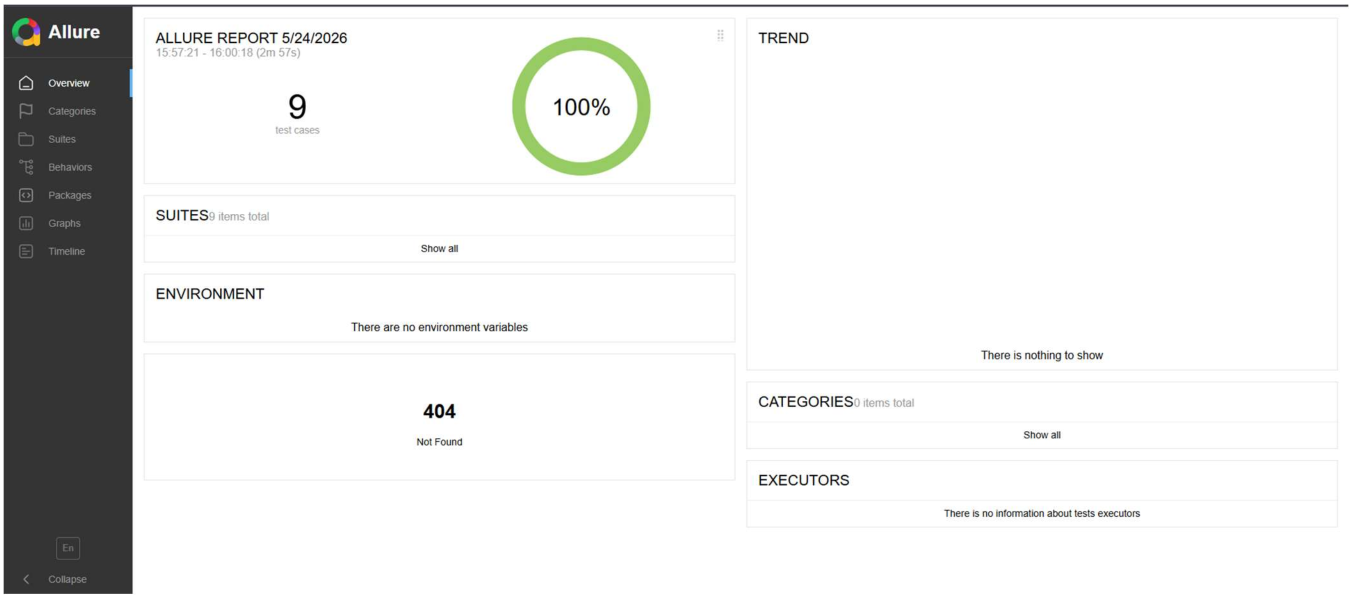

Figure B9: Selenium BDD evidence – Allure Dashboard Overview.

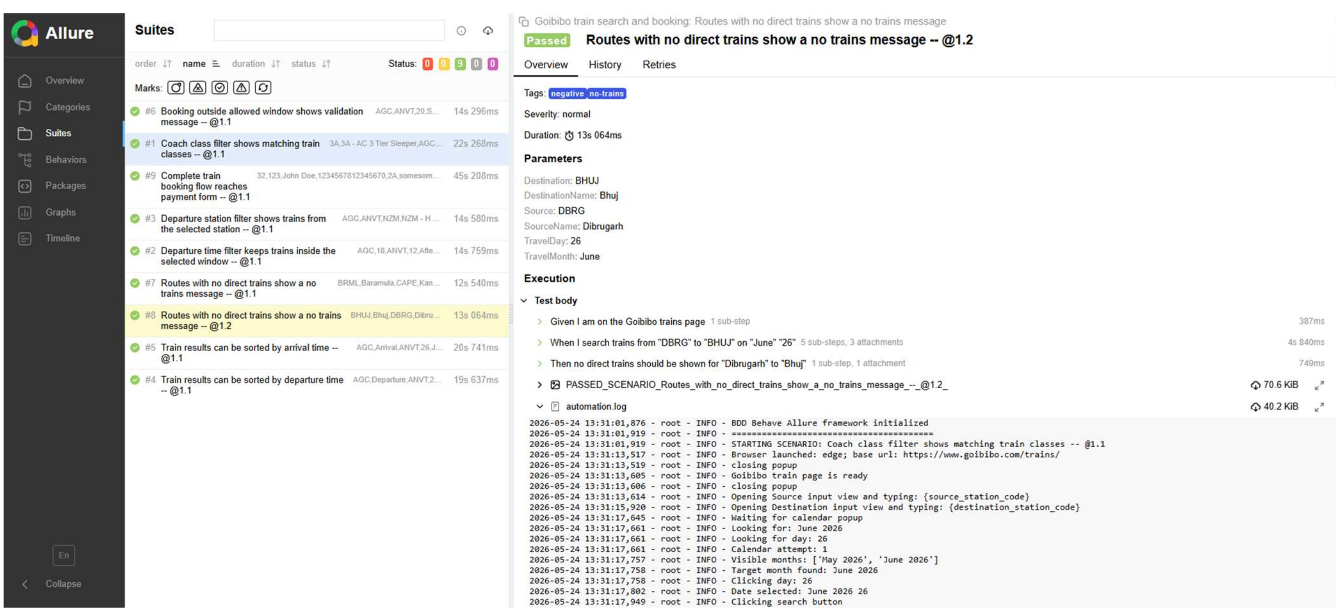

Figure B10: BDD evidence – Test Suits with Steps and Screenshots.

6.7 BDD Execution Summary

Metric	Result
Stored Allure passed scenario results	9
Stored Allure failed scenario results	0
Stored Allure broken scenario results	0
Overall BDD status	Passed based on available Allure result files

7. Common Reporting, Logging, and Screenshot Strategy

- Both frameworks use Allure reporting as the main execution evidence source.
- Screenshots are captured during important page transitions and validations.
- Failure screenshots are captured automatically to simplify debugging.
- The automation.log file records execution progress, selected inputs, validations, and failures.
- Reports make the project suitable for demonstration because each test result can be traced to screenshots and step logs.

8. Common Pytest vs BDD Comparison

Area	Selenium Pytest POM	Behave BDD
Primary Style	Code-driven testing using Python test functions.	Business-readable testing using Gherkin scenarios.
Data Handling	CSV files for functional cases and Excel for E2E booking flow.	Scenario Outline example tables inside feature file.
Best Audience	Automation testers and developers.	Testers, trainers, reviewers, and business stakeholders.
Reporting	Allure Pytest and HTML reports.	Allure Behave with logs and screenshots.
Maintainability	Reusable page objects and utilities.	Reusable page objects plus readable feature files.

9. Common Strengths of the Project

- Uses Page Object Model consistently for better maintainability.
- Covers positive, negative, sorting, filter, and end-to-end workflows.
- Uses Allure reports with screenshots and logs for professional evidence.
- BDD version improves readability and removes external data files by using Scenario Outlines.
- Runner file supports convenient execution, tag filtering, cleanup, and report serving.
- Uses Selenium waits and expected conditions instead of hard-coded time.sleep synchronization.

10. Common Limitations and Future Enhancements

- Goibibo is a live website, so UI changes, dynamic popups, or locator changes may require maintenance.
- Real payment is not completed because the automation stops safely at test payment form validation.
- Future enhancement can add CI/CD execution and automatic Allure publishing.
- Future enhancement can add cross-browser matrix execution for Chrome and Edge.
- Future enhancement can add environment metadata in Allure report for browser, OS, and base URL.

11. Common Conclusion

The Goibibo Train Booking Automation project successfully demonstrates two strong automation frameworks for the same application workflow. The Selenium Pytest POM framework proves technical automation capability through reusable page classes, parametrized tests, and data-driven execution. The Behave BDD framework improves readability by translating the same intent into Gherkin scenarios and Scenario Outlines.

Together, the two frameworks show practical understanding of Selenium WebDriver, Page Object Model, test design, positive and negative validation, end-to-end testing, logging, screenshots, and Allure reporting. The project is suitable for capstone evaluation and live presentation because both code-level implementation and business-readable BDD documentation are available.